

Lab 8: Optimization

Refresher on Convex Functions and Sets

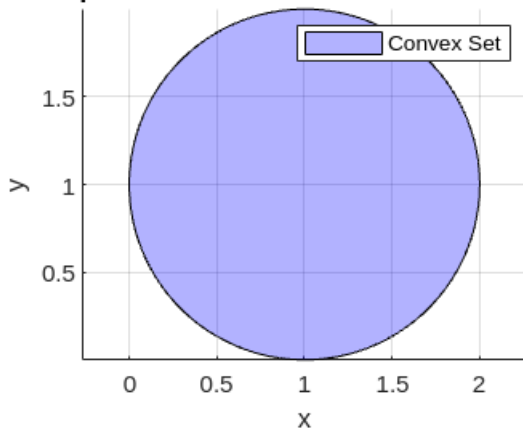
%% Convex Functions and Convex Sets

A set in a vector space is called convex if, for every pair of points within the set, every point on the straight line segment that joins them is also within the set. A function is convex on an interval if the line segment between any two points on the graph of the function lies above or on the graph in that interval.

% Plotting a Convex Set

```
figure;
hold on;
theta = linspace(0, 2*pi, 100);
x = cos(theta) + 1; % Shifting the circle to make it clearly within positive
quadrant
y = sin(theta) + 1; % Shifting the circle to make it clearly within positive
quadrant
fill(x, y, 'b', 'FaceAlpha', 0.3);
title('Example of a Convex Set: A Shifted Unit Circle');
xlabel('x');
ylabel('y');
axis equal;
grid on;
legend('Convex Set');
```

Example of a Convex Set: A Shifted Unit Circle



% Plotting a Non-Convex Set: Crescent Shape

```
figure;
theta = linspace(0, 2*pi, 100);
x_outer = cos(theta) * 1.2; % Outer circle with radius 1.2
y_outer = sin(theta) * 1.2;
```

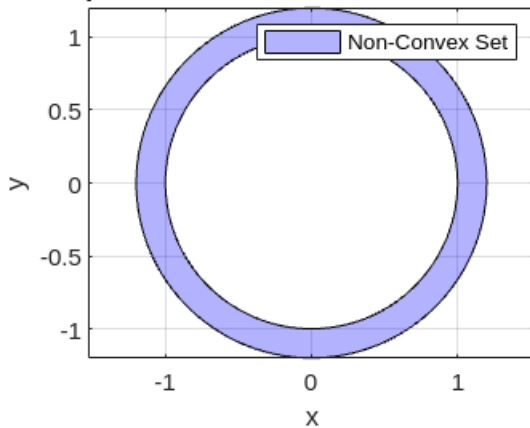
```

x_inner = cos(theta); % Inner circle with radius 1
y_inner = sin(theta);

fill(x_outer, y_outer, 'b', 'FaceAlpha', 0.3);
hold on;
fill(x_inner, y_inner, 'w', 'FaceAlpha', 1); % White fill to create the
crescent shape
title('Example of a Non-Convex Set: Crescent Shape');
xlabel('x');
ylabel('y');
axis equal;
grid on;
legend('Non-Convex Set');

```

Example of a Non-Convex Set: Crescent Shape

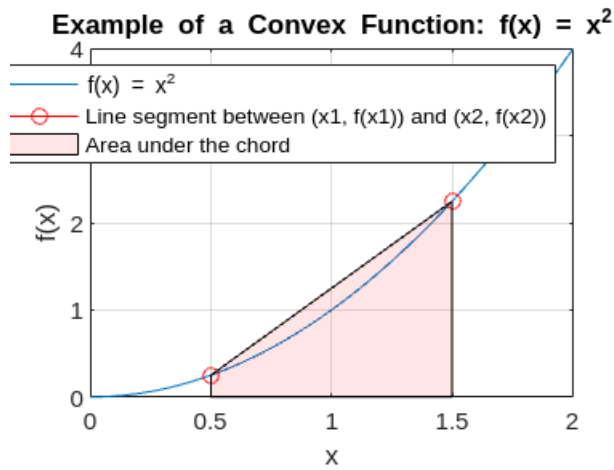


A convex function $f(x)$ holds the property that for any two points x_1 and x_2 , and any t in $[0, 1]$, the following inequality is satisfied: $f(t \cdot x_1 + (1-t) \cdot x_2) \leq t \cdot f(x_1) + (1-t) \cdot f(x_2)$. This property is illustrated below.

```

% Plotting a Convex Function
figure;
f = @(x) x.^2;
fplot(f, [0, 2]);
hold on;
x1 = 0.5; x2 = 1.5;
plot([x1, x2], [f(x1), f(x2)], 'ro-');
fill([x1 x1 x2 x2], [0 f(x1) f(x2) 0], 'r', 'FaceAlpha', 0.1);
title('Example of a Convex Function: f(x) = x^2');
xlabel('x');
ylabel('f(x)');
grid on;
legend('f(x) = x^2', 'Line segment between (x1, f(x1)) and (x2, f(x2))',
'Area under the chord');

```

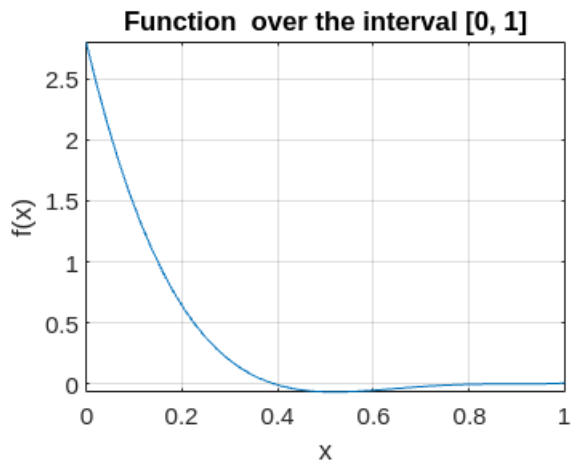


Problem 1: Refresher on Golden section search

Now, let's try to solve this optimization problem that we covered in class using Golden section search.

```
% Define a convex function
f = @(x) 10*(x-0.893).^4 + 5*(x-0.893).^3;

% Plot the function
figure;
fplot(f, [0, 1]);
grid on;
title('Function over the interval [0, 1]');
xlabel('x');
ylabel('f(x)');
```



```
% Initialize the search interval and tolerance
a = 0; b = 1;
tol = 1e-5;
maxIter = 10;

% Perform golden section search to find the minimum
```

```
fprintf('\nNow, let's find the minimum using golden section search.\n');
```

Now, let's find the minimum using golden section search.

a) Implement **goldenSectionSearch(f, a, b, tol, maxIter)** at the bottom of the script.

```
% c = goldenSectionSearch(f, a, b, tol, maxIter);
```

Problem 2: Optimization of a Tension/Compression Spring Design using **fmincon**

The goal of this problem is to introduce the function **fmincon** for the optimization of a structural engineering problem, specifically minimize the weight of a coil spring under a constant tension/compression load with mechanical constraints. The image of the coil spring is shown as follow:

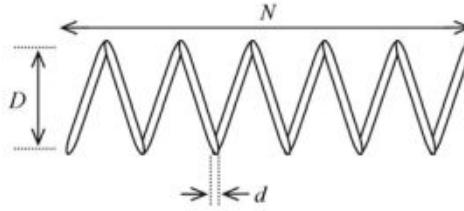


Fig. 12.9 Tension-compression spring

The weight of the spring is to be minimized, subject to constraints on the minimum deflection (g_1), shear (g_2), and surge frequency (g_3), and to limits on the outside diameter (g_4) as shown in the following figure:

$$\text{Minimize: } f(N, D, d) = (N + 2) \times D d^2 \quad (12.24)$$

Subject to:

$$g_1 = 1 - \frac{D^3 N}{71785 d^4} \leq 0 \quad (12.25)$$

$$g_2 = \frac{4D^2 - Dd}{12566(Dd^3 - d^4)} + \frac{1}{5108d^2} - 1 \leq 0 \quad (12.26)$$

$$g_3 = 1 - \frac{140.45d}{D^2 N} \leq 0 \quad (12.27)$$

$$g_4 = \frac{D + d}{1.5} - 1 \leq 0 \quad (12.28)$$

where the domain of interest of each design variable being: $0.05 \leq d \leq 1$, $0.25 \leq D \leq 1.3$ and $2 \leq N \leq 15$.

The above coil's image and problem formulation is from these two papers:

- Gandomi, A. H., & Yang, X. S. (2011). Benchmark problems in structural optimization. In *Computational optimization, methods and algorithms* (pp. 259-281).
- Berlin, Heidelberg: Springer Berlin Heidelberg. Aragón, V. S., Esquivel, S. C., & Coello, C. A. C. (2010). A modified version of a T-Cell Algorithm for constrained optimization problems. *International Journal for Numerical Methods in Engineering*, 84(3), 351-378.

a) Define the objective based on the problem formulation provided.

```
clc; clear all; close all;

% Objective function (x1: N, x2: D, x3: d)
% obj = @(x) ...;
```

b) Define all the constraint functions based on the problem formulation provided by completing the function **[c, ceq] = SpringConstraints(x)**

c) Set up the initial guess of x and the upper and lower bounds for each design variable.

```
% Initial guess
% x0 = [...];

% Lower and upper bounds
% lb = [...];
% ub = [...];
```

d) Set options for **fmincon** using **optimoptions** function

```
% Set options for fmincon
% options = optimoptions(...);
```

e) Solve the problem and display the optimal values for N, D, d and the objective value.

```
% Solve the problem

% [x, fval] = fmincon(obj, x0, [], [], [], [], lb, ub, @SpringConstraints,
options);

% Display the optimal values
% disp('Optimal values:');
% disp(['N: ', num2str(x(1))]);
% disp(['D: ', num2str(x(2))]);
% disp(['d: ', num2str(x(3))]);
% disp(['Objective function value: ', num2str(fval)]);
```

However, in our original formulation, we didn't enforce N to be an integer. Practically, N must be an integer as it represents the number of coils. Next, we will enforce this by fixing N to be an integer (either 11 or 12) and checking what other values we get for d and D. We do this by using constraint matrices.

So let us find the value of D and d, if N is set to 11 or 12?

```
% Solve the problem again

% [x, fval] = fmincon(...);
```

Problem 3: Optimization of the solar heat exchange problem

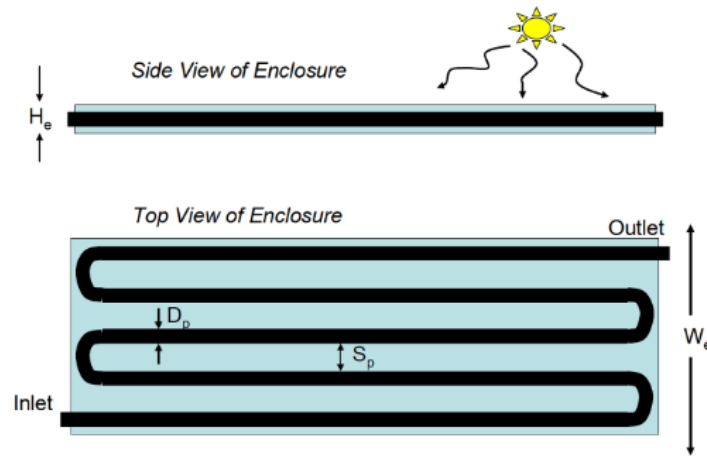


Figure S5. Solar heat exchanger problem schematic.

Table S3. Solar heat exchanger design variables.

Design Variable	Symbol	Sample Range
Pipe diameter	D_p	2 to 12cm
Pipe segments	N_p	9 to 40

This design problem seeks to create a solar energy absorber to heat water. It is a multiobjective problem with a mix of discrete and continuous variables and functional dependence between variables. The device heats water by pumping it through a black pipe that is exposed to direct sunlight. To fit into a rectangular enclosure, the pipe must bend and reverse direction multiple times in such a way that each pipe length between bends is the same.

The two objectives of this problem are to **minimize the head loss** due to friction in the pipe. As a result of the assumptions in the problem, the two independent variables are pipe diameter and number of pipe loops. The number of pipe loops is discrete and must be integer-valued. We have constructed the objective and constraints function as listed at the bottom.

The objective function is **output = Solar_hex(X)**, taking in the design variables $X = [D_p, N_p]$ and return the value of head loss.

The constraints function is **[c, ceq] = Solar_hex_constraints(X)**. The two main inequality constraints are the volume of the enclosure $\leq 0.4 \text{ m}^3$ and the length of enclosure $\leq 1.5 \text{ m}$. There is no equality constraint.

The problem is from this paper: C. Sharpe, T. Wiest, P. Wang, and C. C. Seepersad, "A Comparative Evaluation of Supervised Machine Learning Classification Techniques for Engineering Design Applications," *Journal of Mechanical Design*, vol. 141, no. 12, Oct. 2019, doi: 10.1115/1.4044524.

a) Set up the initial guess, upper bounds and lower bounds. Use **fmincon()** to solve this optimization problem:

```
% Define the objective function
objective = @Solar_hex;

% Define the constraints function
constraints = @Solar_hex_constraints;

% Initial guess
% x0 = [...];

% Lower and upper bounds, assuming all variables are positive
% lb = [...];
% ub = [...];

% Set up optimization
% options = optimoptions(...);

% Run the optimizer
% [optimal_X, optimal_fval] = fmincon(...);

% Display the results
% fprintf('Optimal X: %f (pipe diameter), %f (number of pipes)\n',
optimal_X(1), optimal_X(2));
% fprintf('Optimal function value: %f\n', optimal_fval);
```

b) Now, use the built-in genetic algorithm (**ga**) for this optimization problem and observed how ga's result is different from the previous result.

```
clc

% Define the objective function
objective = @Solar_hex;

% Define the constraints function
constraints = @Solar_hex_constraints;

% Initial guess
% x0 = [...];

% Lower and upper bounds, assuming all variables are positive
% lb = [...];
% ub = [...];

% Define options for the genetic algorithm
```

```

% options = optimoptions(...);

% Run the genetic algorithm
% [optimal_X, optimal_fval, exitFlag, output, population, scores] =
ga(objective, 2, [], [], [], [], lb, ub, constraints, options);

% Display the results
% fprintf('Optimal X: %f (pipe diameter), %f (number of pipes)\n',
optimal_X(1), optimal_X(2));
% fprintf('Optimal function value: %f\n', optimal_fval);

```

```

function c = goldenSectionSearch(f, a, b, tol, maxIter)
    % Define the golden ratio constant ( $\tau = 0.618$ )
    tau = 0.618;

```

```

    ...
end

```

```

function [c, ceq] = SpringConstraints(x)
    ...
end

```

```

function output = Solar_hex(X)
    % These global variables are for plotting purpose
    global iter;
    global X_iter;
    global plotting;

    % Calculating the objective
    m_dot = 0.2;
    q = 1000.0;
    friction = 0.4;
    rho_water = 997.1;
    cp_water = 4200.0;
    clearance = 0.02;
    g = 9.81;

    d_p = X(:,1); % pipe diameter
    n_p = X(:,2); % number of pipes

    s = 8 * friction / (pi^2 * g) .* (m_dot / rho_water) .^ 2 ./ d_p.^5;

    h_enc = d_p + clearance;
    w_enc = d_p .* n_p + d_p .* (n_p - 1) + clearance;

```



```

    l_s = (3 * m_dot * cp_water / q - (pi * d_p .* (3 * d_p / 2 +
clearance / 2) ...
    + (n_p - 1) * pi^2 / 4 .* ((3 * d_p / 2) .^ 2 - (d_p / 2) .^ 2))) ...
    ./ (pi * d_p / 2 .* n_p);

    l_eff = l_s .* n_p + (3 * d_p + clearance) + 50 * d_p .* (n_p - 1);

    h_l = s .* l_eff;
    l_enc = l_s + 3 * d_p + clearance;
    volume = l_enc .* w_enc .* h_enc;

    output = h_l*1e10;

    % Storing the searched X at each iteration for fmincon
    if ~plotting
        X_iter(iter,:) = X;
        iter = iter + 1;
    end
end

```

```

function [c, ceq] = Solar_hex_constraints(X)
    m_dot = 0.2;
    q = 1000.0;
    friction = 0.4;
    rho_water = 997.1;
    cp_water = 4200.0;
    clearance = 0.02;
    g = 9.81;

    d_p = X(:, 1); % pipe diameter
    n_p = X(:, 2); % number of pipes

    s = 8 * friction / (pi^2 * g) .* (m_dot / rho_water) .^ 2 ./ d_p.^5;

    h_enc = d_p + clearance;
    w_enc = d_p .* n_p + d_p .* (n_p - 1) + clearance;

    l_s = (3 * m_dot * cp_water / q - (pi * d_p .* (3 * d_p / 2 +
clearance / 2) ...
    + (n_p - 1) * pi^2 / 4 .* ((3 * d_p / 2) .^ 2 - (d_p / 2) .^ 2))) ...
    ./ (pi * d_p / 2 .* n_p);

    l_eff = l_s .* n_p + (3 * d_p + clearance) + 50 * d_p .* (n_p - 1);

    h_l = s .* l_eff;
    l_enc = l_s + 3 * d_p + clearance;
    volume = l_enc .* w_enc .* h_enc;

```

```
c1 = volume - 0.4;  
c2 = l_enc - 1.5;  
  
output = [c1, c2];  
c = output;  
ceq=[];  
end
```